MONITOR Z · L00.1

Clase 1 — El demonio en Unix: la idea que ya conocés

Cuaderno de estudio · versión del 2026-10-26

Margen derecho y renglones libres para tus notas. Las láminas son las mismas de la carpeta **_visual**. Si una lección se corrige, se regenera este cuaderno con la fecha nueva.

L00.1 — Clase 1 · El demonio en Unix: la idea que ya conocés

Temario de la L00: 1. El demonio en Unix · 2. La misma idea en z/OS: STC, JES y address spaces · 3. Por qué no alcanza con «preguntar cada tanto» · 4. Las piezas del monitor, y dónde estamos hoy.

Objetivo de esta clase: saber qué hace que un programa sea un *demonio* y reconocer cada rasgo en dos programas que ya corriste: el **puente** de la PC y **ZTKMON** en DESA. Sobre todo el rasgo principal: **esperar sin gastar CPU**.

> 📚 Fuentes: - W. R. Stevens y S. Rago, *Advanced Programming in the UNIX Environment*, cap. 13 «Daemon Processes», y la página `daemon(7)` del manual de Linux: qué es un demonio y sus rasgos. - *The Open Group Base Specifications*, `select()`: esperar varios descriptores con un plazo. - `plataforma_cierres/vivo/puente.py`: `correr()` (líneas 242–254), `latidos()` (143–150) y la lectura (154–157). - `monitor_z/asm/ZTKMON.asm` (v0.9): la cabecera (líneas 43–47) y la rutina `SERVIR`. - El listado del ensamblado **JOB34775** (sentencias 11522–11554), el SYSPRINT y el JESYSMSG del monitor **JOB34879** (DESA, 2026-10-22) y el JCL `ZTKMONVR`. - IDEA-005 en `IDEAS_cierres_produccion.md`: el `sem_timedwait` de SOPES2.

1. Qué es un demonio: seis rasgos

Analogía: el **guardia nocturno** de un edificio. Nadie lo llama para que empiece: entra con su turno. Se queda toda la noche. Casi todo el tiempo está sentado mirando y no corre por los pasillos. Cuando suena algo, hace una ronda corta y vuelve a la silla. Obedece al jefe de turno y anota todo en el libro de novedades.

#	Rasgo	En Unix	En el puente (PC)	En ZTKMON hoy (DESA)
1	**Arranca sin terminal**	lo levanta el sistema (`systemd`, `init`), o se separa de la terminal con `fork` + `setsid`	❌ lo arrancás vos en una consola: `python puente.py --host … --puerto 8010`	❌ lo sometés vos: el job `ZTKMONR` (miembro `ZTKMONVR`). En la L07 pasa a STC

2	**Vive mucho**	mientras el sistema esté arriba	✅ `correr()` es un `while True` (línea 245)	🟡 **30 minutos**, por el PARM `'ZTKD* 5 30 8010'`. JOB34879: `+HOLA … T=11:37:14` → `+FIN ZTKMON T=12:07:16`
3	**Espera casi siempre**	`sleep`, `select` / `poll`, un semáforo	✅ `await r.read(8192)` y `await asyncio.sleep(30)`	✅ `SELECT` con plazo de 5 s (sección 3)
4	**Hace un trabajo corto y vuelve a esperar**	atiende un pedido y vuelve al `select`	✅ arma el evento y lo pone en la cola	✅ una vuelta: **6,7 a 6,95 ms** de CPU (`+DIAG COSTO`, JOB34879)
5	**Obedece desde afuera**	señales: `SIGTERM` para parar, `SIGHUP` para releer la configuración	❌ no atiende señales: se para con Ctrl+C	🟡 contesta pedidos por el socket (`PING`, `ACT`, `ADIOS` …), pero **ninguno lo para**. Termina por reloj. `F` y `P` del operador llegan en la L07
6	**Deja rastro**	`syslog`	✅ `print(…, flush=True)` a la consola	✅ el SYSPRINT: `+HOLA`, `+EVT`, `+DIAG` y un `+PONG` cada 12 vueltas (31 en JOB34879)

Lo que dice la tabla: ninguno de los dos es todavía un demonio completo. Los dos esperan bien (rasgo 3), y ese es el rasgo difícil. Los rasgos 1 y 5 son los que trae la L07, cuando ZTKMON pase a STC.

> Que el puente no atiende señales se ve en el código: ningún `try` captura `KeyboardInterrupt` y no hay ningún `mandar('ADIOS')`, aunque la cabecera lo nombre (línea 18). Al cortarlo, el host ve que el cliente se fue sin despedirse.



Lámina L00-09_seis_rasgos

2. El rasgo que importa: esperar sin gastar

Un demonio **malo** pregunta en un bucle sin pausa:

```
while (1) { preguntar(); }      /* nunca suelta la CPU */
```

Un demonio **bueno** le dice al sistema operativo «despertame cuando pase algo, o en 5 segundos»:

```
while (1) { esperar(evento, 5 s); atender(); }
```

Qué pasa mientras espera: el sistema operativo **lo saca de la CPU** y lo anota en una cola de espera. No cuenta tiempo de CPU porque no corre: no es que corra despacio. Cuando llega el evento o vence el plazo, lo vuelve a poner en la cola de listos. En Unix se ve en `ps` con el estado `S` (*sleeping*). En z/OS la tarea (TCB) queda en *wait*, que es la L02.

Medido en ZTKMON (JOB34879, 2026-10-22):

Dato	Valor	De dónde sale

Cuánto vivió	**1.800 s** (11:37:14 → 12:07:16)	`+HOLA` y `+FIN ZTKMON` del SYSPRINT
Vueltas	**360** (cada 5 s)	la última `+DIAG COSTO VUELTAS=00360`
CPU de todo el paso	**2,55 s** de TCB y 0,01 s de SRB	`IEF032I STEP/MONITOR … CPU: 0 HR 00 MIN 02.55 SEC` del JESYSMSG
Fracción de una CPU	**≈ 0,14 %** (2,56 / 1.800)	cuenta sobre los dos datos anteriores
CPU por vuelta	**6,71 a 6,95 ms** (de 402.344 a 417.090 µs cada 60 vueltas)	las 6 líneas `+DIAG COSTO`
Tiempo esperando	**≈ 99,86 %** de cada vuelta (6,8 ms de cada 5.000)	cuenta

El contraste (cálculo, no medido): si ZTKMON no esperara, cada vuelta seguiría a la anterior sin pausa y el paso usaría una CPU entera los 1.800 s, unas **700 veces** lo que gastó. No se midió y no hay que medirlo: en un LPAR compartido eso le quita CPU a los jobs del cierre.

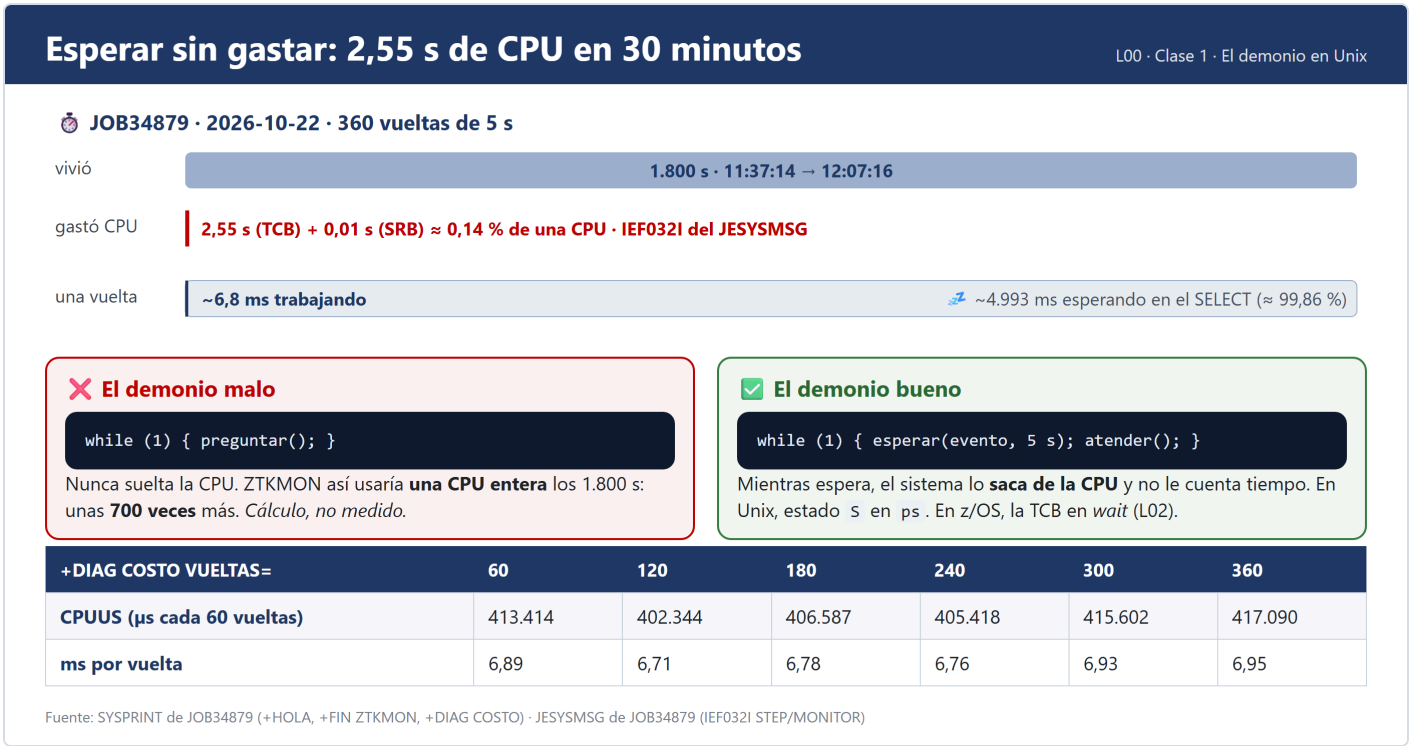


Lámina L00-10_esperar_sin_gastar

3. Las tres maneras de esperar, y cuál usa ZTKMON

Manera	Unix	Te despierta…	En z/OS	Dónde aparece en el proyecto
⏰ **Dormir un rato**	`sleep(5)`	el reloj	`STIMER WAIT,BINTVL=…`	el respaldo de ZTKMON si el `SELECT` falla (sentencia 11551). Teoría en L02
🚦 **Semáforo**	`sem_wait` / `sem_post`, `sem_timedwait` (SOPES2)	**otro** que avisa	**ECB** con `WAIT` y `POST`	el nivel C (ENF 70, L08). Teoría en L02
📡 **Varios a la vez, con plazo**	`select()` / `poll()`	el **primero** que llegue: un dato, una conexión o el plazo	`SELECT` de la API de sockets (`EZASOKET`)	**todo el bucle de ZTKMON** (sentencia 11522)

ZTKMON usa la tercera. La cabecera lo dice (líneas 43–47): *«en vez de STIMER WAIT, cada vuelta espera con SELECT (plazo = <segundos>) sobre el socket que importa… si no, a los <segundos> se escanea igual»*. Así una sola espera sirve para dos cosas: despertar cada 5 s para mirar a JES, y despertar **ya** si el puente manda un pedido.

Los bytes (JOB34775):

```
001038 00002010   DC A(FSELECT)                  ← la lista de parámetr
001040 000020C8   DC A(PLAZO)                    ← el plazo: los 5 s de
 …
001060 800020A4   DC A(RETCODE+X'80000000')      ← el último lleva el b
001064 58F0 B02C  L     15,IHB0058B              ← la dirección de EZAS
001068 05EF       BALR  14,15                    ← acá el programa SE D
00106A BF2F A0A4  ICM   R2,15,RETCODE            ← y acá sigue al despe
00106E 4780 B1F4  BZ    SVFIN                    ← 0: venció el plazo,
001072 4720 B092  BP    SVHAY                    ← > 0: llegó una conex
001076 …          (negativo: el SELECT falló)    → TRAZA y STIMER WAIT,
```

El detalle que muestra oficio: si el `SELECT` fallara al instante en cada vuelta, el bucle giraría sin dormir y sería el demonio malo de la sección 2. Por eso la rama de error hace `STIMER WAIT`. El comentario de `SERVIR` lo dice así: *«Si el SELECT falla, espera con STIMER para no girar en vacío»*. Esa rama **no se ejecutó** en JOB34879: no hay ninguna línea `+DIAG SOCK FUNCION=SELECT` con error en su SYSPRINT.

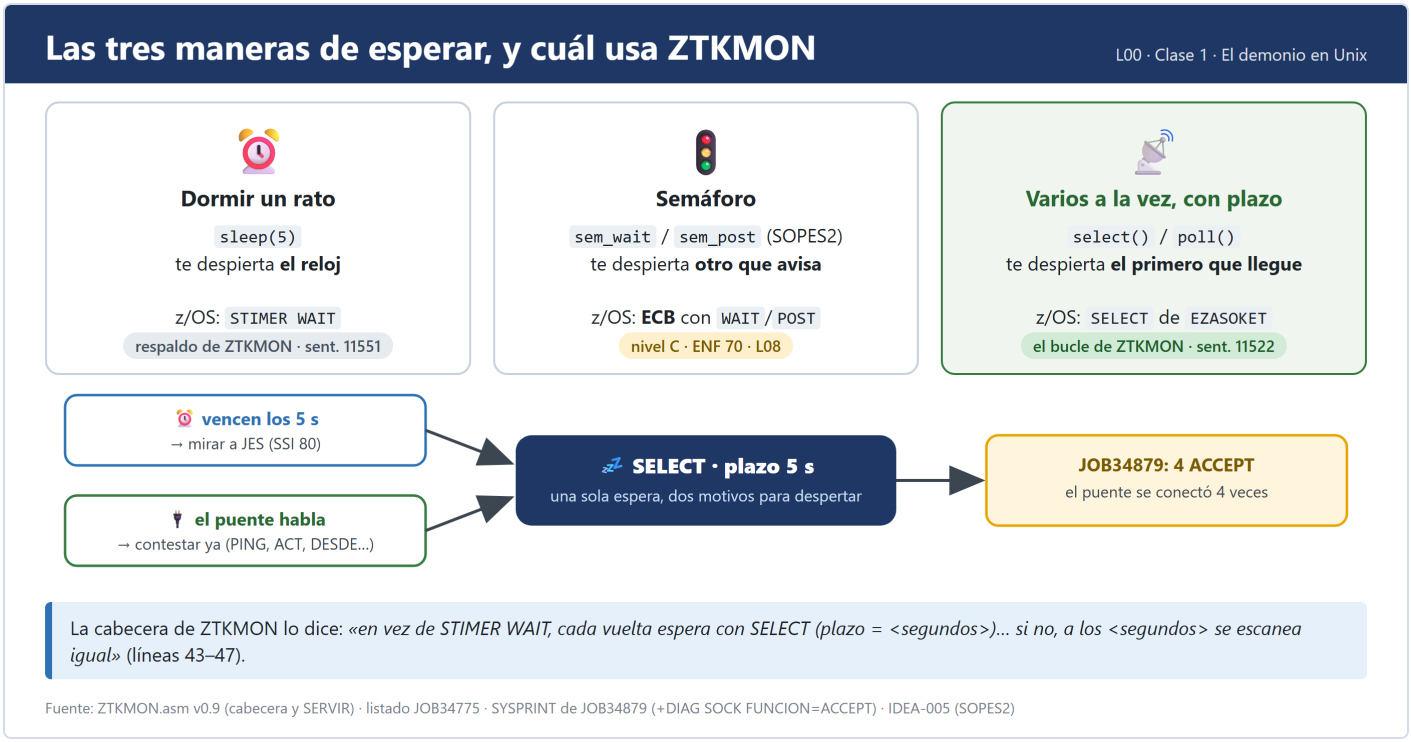


Lámina L00-11_tres_maneras

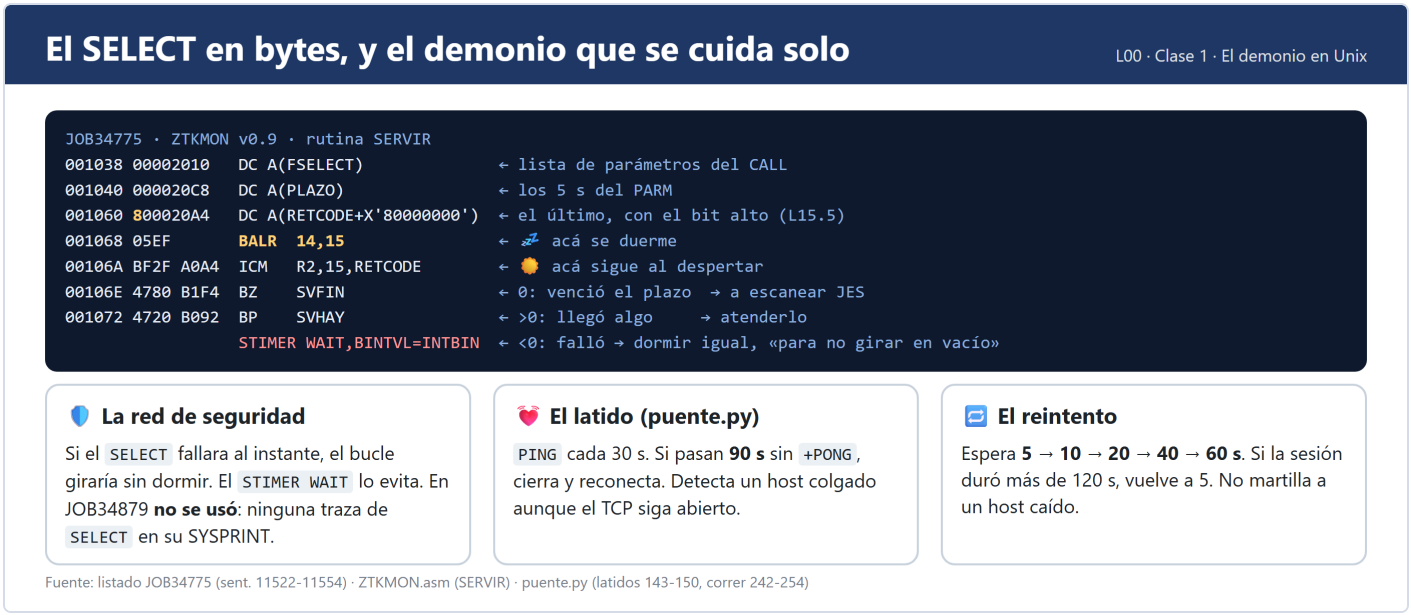


Lámina L00-12_el_select_en_bytes

4. Un buen demonio se cuida solo

Esperar bien no alcanza: si la otra punta se calla o se corta, el demonio se tiene que dar cuenta y levantarse solo. El puente lo hace de dos maneras:

Mecanismo	Código (`puente.py`)	Qué pasa
💓 **Latido**	`latidos()`: `sleep(30)` → `PING`; si pasan 90 s sin `+PONG`, cierra (líneas 143–150)	detecta un host colgado aunque el TCP siga «abierto»
🔁 **Reintento con espera creciente**	`correr()`: 5, 10, 20, 40 y 60 s como tope; si la sesión anduvo más de 120 s, vuelve a 5 (líneas 242–254)	no martilla al host mientras está caído, y vuelve rápido después de un corte corto

Del lado de ZTKMON, el `+PONG` es la respuesta al latido. Que lleve contadores (vueltas, CPU, último SEQ) es la **A10**.

5. Qué cambia en z/OS (la clase 2)

La idea es la misma, pero cambian los nombres y **quién arranca y para** al demonio:

Unix	z/OS	Clase
proceso	address space, con sus tareas (TCB)	2
demonio que arranca con el sistema	**STC**: `S ZTKMON`, sin ocupar un iniciador	2
proceso largo lanzado a mano	**job batch largo**: lo que es ZTKMONR hoy	2
`SIGTERM` / `SIGHUP`	`P ZTKMON` / `F ZTKMON,texto`	2
`ps`	preguntarle a JES2 por SSI 80	3

Resumen

Idea	En una línea	Testigo
Demonio	arranca solo, vive mucho, espera, trabaja corto, obedece y anota	Stevens, cap. 13

El rasgo difícil	esperar **sin** gastar CPU: el sistema lo saca de la CPU	JOB34879: 2,55 s en 1.800 s
Cómo espera ZTKMON	`SELECT` con plazo: JES cada 5 s, o el puente apenas habla	sentencia 11522, `BALR 14,15` en `001068`
La red de seguridad	si el `SELECT` falla, `STIMER WAIT` para no girar en vacío	sentencia 11551
Lo que falta para ser demonio de verdad	arrancar solo y obedecer a `P` / `F`: pasar a STC	L07
Cuidarse solo	latido de 30/90 s y reintento de 5 a 60 s	`puente.py` 143–150 y 242–254

Preguntas de comprobación

1. JOB34879 gastó 2,55 s de CPU en 1.800 s. Si el `SELECT` fallara **siempre** al instante y se borrara la rama del

`STIMER WAIT` (sentencia 11551), ¿qué pasaría con ese número, y por qué?

1. De los seis rasgos, ¿cuáles cumple hoy ZTKMON y cuáles le faltan? Para los que faltan, decí qué los trae.

Mis respuestas

Relacionado: *L00 - Que es un demonio en z-OS y que vamos a construir* · *L02 - Dormir sin gastar CPU - STIMER, WAIT, POST y ECB* · *L06 - Sockets en z-OS* · siguiente: L00.2.

Notas